Verificación de las demostraciones en Lean 4

Proyecto: Few biological parameters suffice to reproduce measured dynamics (paquete de reproducibilidad, carpeta `lean_proofs/`). Verificación hecha el 16 de octubre de 2026 en una copia limpia del repositorio (commit `b64f2c7`), en un servidor Linux de 88 núcleos, con Lean 4.23.0 y Mathlib v4.23.0.

1. La pregunta: ¿las demostraciones funcionan?

Una demostración formal puede fallar de dos maneras muy distintas.

Tipo de fallo	Qué significa	Qué tan probable es
La demostración es incorrecta	Algún paso lógico no es válido, o se dejó un hueco sin demostrar (sorry), o se añadió una suposición extra (axiom).	Muy improbable si el núcleo de Lean la acepta y no hay sorry ni axiomas extra: Lean revisa cada paso.
El enunciado no dice lo que se cree	El teorema es cierto, pero sus definiciones difieren de las del paper, sus hipótesis no se pueden cumplir nunca (teorema vacío) o su conclusión es trivial.	Es el riesgo real. No lo detecta el compilador: hay que comprobarlo aparte.

Las comprobaciones de las secciones 2 y 3 atacan el primer tipo de fallo; las de la sección 4, el segundo.

2. ¿Compila todo, desde cero, sin huecos?

Qué se hizo. Se clonó el repositorio en una carpeta nueva (como en una máquina recién instalada), se descargó Mathlib precompilada y se compilaron con `./verify.sh` los 32 módulos de teoría, el archivo de testigos `NonVacuity.lean` y los 28 modelos del benchmark (Chen y Froehlich divididos en partes: 46 módulos de modelos).

Por qué. Compilar en la máquina propia puede ocultar problemas (archivos compilados antiguos, versiones distintas). Una compilación desde cero es lo que hará un revisor.

Resultado. 2.658 tareas de compilación sin ningún error, en 18 minutos. La búsqueda de `sorry` y `axiom` en las fuentes no encontró ninguno. Los únicos mensajes son avisos de estilo (variables sin usar), que no afectan a la validez.

```
1/4  Looking for 'sorry' and 'axiom' declarations...
Build completed successfully (2658 jobs).
Done: every theorem and every model uses only [propext, Classical.choice, Quot.sound] (or a subset).
```

3. ¿Qué suposiciones usa cada teorema?

Qué se hizo. Se pidió a Lean, con `#print axioms`, la lista de axiomas de los que depende cada uno de los 309 teoremas y lemas, y el teorema final de cada uno de los 28 modelos. Además se pasó todo por lean4checker, un segundo verificador independiente que vuelve a revisar cada declaración ya compilada.

Por qué. Si alguna demostración usara `sorry` (aunque fuera de forma indirecta, a través de otro lema) o un axioma añadido, aparecería en esta lista. Los tres axiomas estándar (`propext`, `Classical.choice`, `Quot.sound`) son los de las matemáticas usuales y los usa toda Mathlib. lean4checker descarta que un error en el proceso de compilación haya dejado pasar algo.

Comprobación	Resultado
Axiomas de los teoremas y lemas (CheckAxioms.lean)	309 de 309 informados; 308 dependen de [propext, Classical.choice, Quot.sound] y 1 de [propext, Quot.sound]; ninguno de otro axioma
Axiomas del teorema final de cada modelo (CheckModelAxioms.lean)	28 de 28 informados, todos solo con los axiomas estándar
lean4checker sobre la teoría y los testigos	33 de 33 módulos aceptados
lean4checker sobre los modelos	46 de 46 módulos aceptados
Control negativo: un teorema falso, 1 = 2, demostrado con sorry	detectado: Lean informa que depende de sorryAx

El control negativo es importante: demuestra que la comprobación de axiomas sí detecta un hueco cuando lo hay, de modo que el resultado limpio de los 309 teoremas no se debe a que la comprobación no funcione.

```
CheckAxioms.lean: 309 of 309 declarations reported, 0 with non-standard axioms
CheckModelAxioms.lean: 28 of 28 declarations reported, 0 with non-standard axioms
```

4. ¿Los teoremas dicen lo que el paper afirma?

4.1 Testigos de no vacuidad. Un teorema cuyas hipótesis no se pueden cumplir nunca es cierto pero no dice nada. El archivo `NonVacuity.lean` construye, para cada teorema central, un ejemplo concreto en el que todas las hipótesis están demostradas, y para las propiedades definidas, un ejemplo en el que fallan (para descartar que se cumplan siempre). Lean verifica cada ejemplo igual que los teoremas.

Teorema del paper	Testigo verificado	Ejemplo negativo
Teorema 1: umbral $\tau = \sqrt{(1 - c^2)}$	dos vectores concretos de R^2	—
Teorema 2: eliminación hacia atrás (las dos partes)	regla $\|S\| \geq 2$ sobre tres parámetros, con regla de eliminación concreta; resultado admisible, mínimo y sin subconjunto propio admisible	—
Filtros κ y VIF de la selección voraz	la identidad cumple $\kappa \leq 1$	la aplicación nula no cumple ni κ ni VIF
Re-estimar más parámetros no aumenta el error	un parámetro con error $\|\theta - 1\|$	—
Estabilidad de mínimos cuadrados (κ)	la identidad en R^2, columnas unitarias	—
Identificabilidad práctica (teorema principal)	residuo no lineal $\delta + \delta^2/100$ en $[-3, 3]$, con la cota del resto demostrada en todo el conjunto	un residuo plano no es identificable
Identificabilidad con resto relativo en la cola (nuevo)	el mismo residuo no lineal, $\eta = 0{,}03$, cola desde $t = 2{,}1$	—

Resultado: todos los ejemplos compilan. Las hipótesis de los teoremas centrales son consistentes, y las definiciones de κ, VIF e «identificable» no son triviales.

4.2 Auditoría de definiciones. Se comparó cada definición de Lean usada por un teorema central con lo que dice el paper y con lo que calcula el código Python (`DEFINITIONS_AUDIT.md`). Lo encontrado:

Punto	Qué se encontró	Consecuencia
κ y VIF	El código centra las columnas (matriz de correlación); los teoremas de mínimos cuadrados de Lean usan columnas de longitud 1 sin centrar.	La garantía de la selección voraz vale igual. La de mínimos cuadrados vale en 30 de 31 subconjuntos (no en Rahman, VIF = 39).
Identificabilidad	La hipótesis sobre el resto no lineal debe cumplirse en todo el conjunto admisible; se había comprobado solo sobre el perfil.	Nuevos teoremas que solo la piden en la cola; comprobada por muestreo en toda la región: se cumple cerca del óptimo (9 de 74 parámetros con factor 1,5).
Minimalidad (Teorema 2, parte 2)	Necesita que la admisibilidad sea monótona; con un optimizador local no está garantizado.	Comprobado en los datos: 0 violaciones en 660 pares de subconjuntos anidados.
Error de primer orden	La cota de Lean es sobre el error absoluto sin re-estimación; la constante de segundo orden nunca se calculó.	Los 6 problemas de primer orden se verificaron con re-estimación no lineal.
Traducción de los modelos	Los modelos SBML se traducen a Lean con números racionales exactos.	Comparados con el simulador (diferencia máxima 10^{-12}); cambiar un número un 1 % se detecta.

Ningún teorema resultó falso. Todos los problemas estaban en el puente entre el teorema y los números del estudio, y quedan documentados en el paper y en el suplemento.

5. Errores encontrados en el script de verificación (corregidos)

Error	Efecto	Corrección
AllModels no estaba declarado como objetivo de compilación	La comprobación de axiomas de los 28 modelos fallaba en silencio y verify.sh terminaba como si todo estuviera bien	AllModels añadido al lakefile; verify.sh se detiene con error si no se informan todas las declaraciones
La búsqueda de sorry/axiom revisaba también Mathlib descargada	Al ejecutar verify.sh por segunda vez fallaba por palabras de comentarios de Mathlib	La búsqueda excluye la carpeta .lake

6. Qué demuestra Lean y qué no

Lean demuestra que cada paso del método es matemáticamente correcto: el umbral, la minimalidad de la eliminación, la corrección de la selección con κ y VIF, la monotonía de la re-estimación, la existencia y regularidad de

las soluciones de 28 modelos y una condición suficiente de identificabilidad. Lean no demuestra los resultados numéricos del estudio (que dos parámetros basten en 31 de 34 modelos, las predicciones o los perfiles): esos se calculan en Python y se apoyan en las pruebas numéricas. Un teorema se aplica a un resultado concreto cuando las cantidades calculadas cumplen sus hipótesis, como indica la Tabla 3 del paper.

7. Cómo repetir la verificación

```
git clone https://github.com/monicammr/archivos.git
cd archivos/reproducibility/lean_proofs
./verify.sh            # compila todo y revisa los axiomas (unos 20 min con muchos núcleos)
lake env lean NonVacuity.lean     # testigos de no vacuidad
# verificador independiente (https://github.com/leanprover/lean4checker, etiqueta v4.23.0):
lake env <ruta>/lean4checker <Módulo>
```

8. Frase para revisores o coautores

«All proofs compile from scratch without sorry or additional axioms (309 theorems and lemmas and 28 benchmark models; only propext, Classical.choice and Quot.sound), are accepted by the independent checker lean4checker, and every central theorem has a machine-checked witness showing that its hypotheses are satisfiable. The correspondence between the Lean definitions, the paper and the code is documented in DEFINITIONS_AUDIT.md.»

Archivos de respaldo en esta carpeta: BUILD_LOG.md, DEFINITIONS_AUDIT.md, NonVacuity.lean, ProfileIdentifiability.lean, verify.sh y lakefile.lean; registros completos en 00_General/registros_servidor (lean_*.log).